

Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial (approx. quadratic growth)

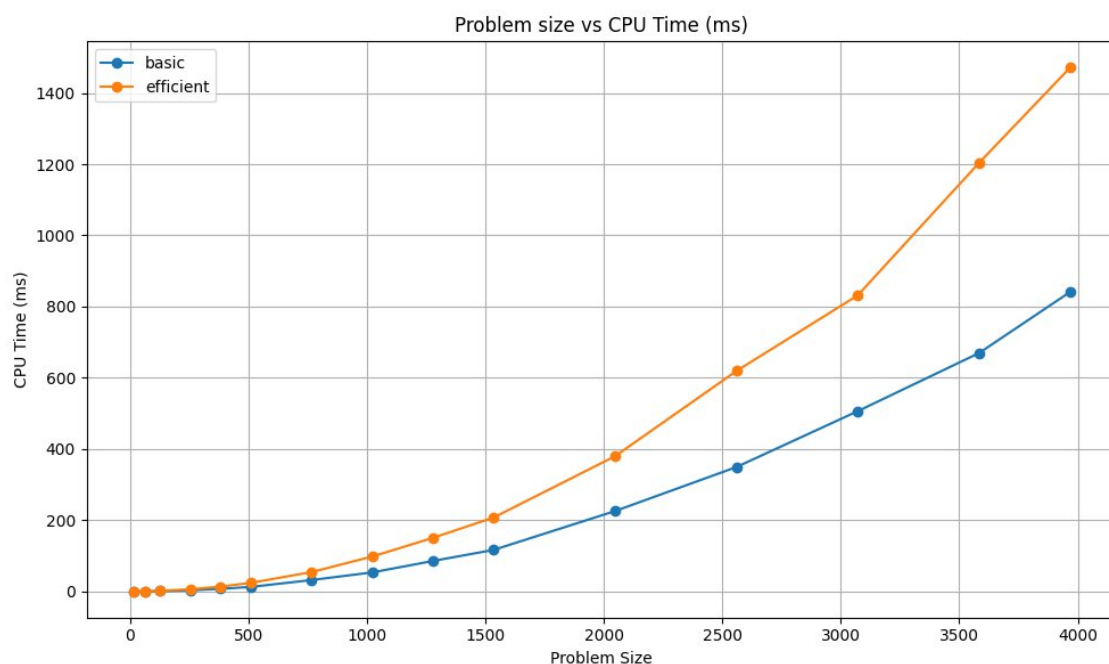
Efficient: Linear (Constant/Flat, space is effectively constant- $O(\min(m,n))$)

Explanation:

- **Basic Dynamic Programming (DP)**
 - As the problem size increases, **space** grows significantly most likely due to the large caches or full matrix.
 - At $M+N \approx 1500$, memory $\approx 1.8 \times 10^4$ KB.
 - At $M+N \approx 3000$, memory $\approx 3.5 \times 10^4$ KB.
 - At $M+N \approx 4000$, memory $\approx 4.5 \times 10^4$ KB.
 - This convex curve reflects $O(m \cdot n) \approx O((M + N)^2)$ behaviour i.e., memory usage accelerates upwards as input size increases.
- **Divide - and - Conquer (Efficient)**
 - **Linear space reduction**- stores only limited rows or score vectors.
 - Memory usage hovers around 1.3×10^4 KB across all tested sizes.
 - The line in the graph is completely horizontal illustrating that the algorithm remains $O(\min(m,n))$, which is effectively constant relative to the rapid growth of the Basic DP.

The Basic DP version's memory consumption diverges sharply (reaching 45,000 KB) beyond the problem size of 2000. The Efficient method stays flat, indicating that it can handle extremely larger sequences without exhausting the memory.

Graph2 - Time vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial (approx. quadratic growth)

Efficient: Polynomial (approx. quadratic, with a large constant factor)

Explanation:

- Basic Dynamic Programming (DP)
 - The algorithm's runtime grows quadratically with the combined length of the input sequences ($M + N$) because it must compute every cell in the $M \times N$ scoring matrix- $O(m \cdot n) \approx O((M + N)^2)$.
 - At $M+N \approx 100$, CPU Time ≈ 0.6 ms.
 - At $M+N \approx 1500$, CPU Time ≈ 100 ms.
 - At $M+N \approx 3600$, CPU Time ≈ 660 ms.
 - At $M+N \approx 4000$, CPU Time ≈ 840 ms.
 - The DP algorithm is **faster** due to its simpler **single-pass table fill** and **single traceback**.
- Divide- and- Conquer (Efficient)
 - Although Time is also $O(m \cdot n)$, the algorithm takes longer in practice because it must re-calculate scores twice at every step to save memory, doubling the workload.
 - At $M+N \approx 100$, CPU Time ≈ 0.9 ms.
 - At $M+N \approx 1500$, CPU Time ≈ 160 ms.
 - At $M+N \approx 3600$, CPU Time ≈ 1200 ms.
 - At $M+N \approx 4000$, CPU Time ≈ 1460 ms.
 - The runtime constant factor **doubles** because every recursive step requires two half-matrix passes to identify the optimal split point.

Both the algorithms show the expected quadratic increase in CPU time as input size grows. However, the **Basic DP algorithm consistently outperforms the Efficient version** due to its simpler table filling method by **25-40%**. While the Divide - and - Conquer algorithm is far better in memory usage, it has a higher runtime for the space that it uses.

Contribution

Equal Contribution

<USC ID/s>: 1477877329, 9603655426, 9977651843, 8768433034, 6517178031